

Chapter 06

DOM 與 JAVASCRIPT 互 動

HORAZON

互動媒體設計

什麼是 JAVASCRIPT?

回顧一下：

- HTML 是 **骨架** (結構)。
- CSS 是 **外觀** (樣式)。
- JavaScript 是 **大腦與肌肉** (行為)。

JS 讓網頁從「靜態文件」變成「動態應用程式」。

它能做什麼？

- 點擊按鈕跳出視窗。
- 檢查表單有沒有填寫。
- **切換深色模式 (Dark Mode)**。
- 從伺服器抓取天氣資料。

認識 DOM (DOCUMENT OBJECT MODEL)

瀏覽器把 HTML 檔案讀進去後，會把它變成一棵 **樹狀結構 (Tree)**。
這棵樹就叫做 **DOM**。

為什麼叫 DOM?

- Document (**文件**)：整個網頁。
- Object (**物件**)：每個標籤 (`<h1>`, `<div>`) 都被轉換成一個「物件」。
- Model (**模型**)：這棵樹的結構模型。

DOM TREE 視覺化

想像 HTML 像家族族譜一樣：

- **document** (祖先)
 - **html** (根)
 - **head**
 - **title**
 - **body**
 - **h1** (標題)
 - **button** (按鈕)
 - **div** (容器)
 - **p** (段落)

JS 就是透過這棵樹，**找到 (Select)** 特定的節點，然後**修改 (Modify)** 它。

第一步：抓取元素 (SELECTION)

要控制某個元素，得先「抓到」它。

我們用 `document.querySelector()` 這個萬能夾子。

```
// 抓取第一個 h1
let title = document.querySelector('h1');

// 抓取 class="btn" 的按鈕
let btn = document.querySelector('.btn');

// 抓取 id="menu" 的選單
let menu = document.querySelector('#menu');
```

變數 (`let`) 就像一個**箱子**，我們把抓到的元素放進去，取名叫 `title`，方便以後呼叫。

第二步：監聽事件 (EVENTS)

抓到元素後，我們要等待使用者的動作。
這就像設下一個觸發機關 (Trigger)。

常見事件：

- `click` (點擊)
- `mouseover` (滑鼠移入)
- `input` (輸入文字)
- `scroll` (捲動頁面)

```
// 當按鈕被點擊 (click) 時，執行後面的功能 (function)
btn.addEventListener('click', function() {
    alert('按鈕被按了！');
});
```

第三步：修改內容與樣式 (MANIPULATION)

事件觸發後，我們可以做什麼？

1. 修改文字 (textContent)

```
title.textContent = "你好，JavaScript！";
```

2. 修改樣式 (style)

```
// 直接改 CSS (不推薦寫太多，偶爾用)  
title.style.color = "red";  
title.style.fontSize = "50px"; // CSS 的 font-size 變 fontSize
```

最佳實踐：切換 CLASS (CLASSLIST)

與其用 JS 一行行改樣式，不如寫好 CSS Class，用 JS 切換開關。
這是最乾淨的做法！

CSS:

```
.dark-mode {  
  background-color: black;  
  color: white;  
}
```

JS:

```
// 切換 class (有就刪掉，沒有就加上)  
body.classList.toggle('dark-mode');
```


瀏覽器開發者工具 (DEVTOOLS)

按 F12 或 右鍵 -> 檢查 (Inspect)。

Console (控制台)

這裡是 JS 的遊樂場。你可以在這裡：

1. 查看 `console.log()` 印出的訊息 (除錯用)。
2. 直接打 JS 程式碼測試。
3. 看到紅色的錯誤訊息 (Error)。

練習： 打開 Console，輸入 `alert('Hi')` 試試看！

變數 (VARIABLES) 與 資料型態

程式裡需要儲存資料。

```
// 字串 (String) - 用引號包起來
```

```
let name = "Horazon";
```

```
// 數字 (Number) - 可以做運算
```

```
let score = 100;
```

```
let price = 50.5;
```

```
// 布林值 (Boolean) - 是非題
```

```
let isLogin = true;
```

```
let isDark = false;
```

邏輯判斷 (LOGIC)

電腦會根據情況做不同決定。

```
let score = 59;

if (score >= 60) {
  // 條件成立 (True)
  console.log("及格!");
  title.style.color = "green";
} else {
  // 條件不成立 (False)
  console.log("不及格...");
  title.style.color = "red";
}
```

實作練習：計數器 (COUNTER)

做一個按鈕，按一下數字加 1。

HTML:

```
<h1 id="count">0</h1>  
<button id="addBtn">加 1</button>
```

JS:

```
let count = 0; // 1. 準備變數  
let text = document.querySelector('#count'); // 2. 抓元素  
let btn = document.querySelector('#addBtn');  
  
btn.addEventListener('click', function() { // 3. 監聽點擊  
    count = count + 1; // 4. 變數加 1  
    text.textContent = count; // 5. 更新畫面  
});
```

實作練習：開關燈 (LIGHT SWITCH)

做一個按鈕，切換網頁亮/暗模式。

CSS:

```
.dark {  
  background: #333;  
  color: #fff;  
}
```

JS:

```
let btn = document.querySelector('button');  
let body = document.querySelector('body');  
  
btn.addEventListener('click', function() {  
  body.classList.toggle('dark'); // 切換 class  
});
```

實作練習：彈出視窗 (MODAL)

點按鈕顯示，點叉叉關閉。

CSS:

```
#modal {  
  display: none; /* 預設隱藏 */  
  position: fixed; /* 蓋在最上面 */  
  /* ...略 (置中樣式) */  
}  
.show { display: block !important; }
```

JS:

```
let modal = document.querySelector('#modal');  
let btn = document.querySelector('#openBtn');  
let close = document.querySelector('#closeBtn');  
  
btn.addEventListener('click', function() {  
  modal.classList.add('show');  
});
```

實作練習：圖片切換 (IMAGE SWITCHER)

點擊小圖，變大圖。

HTML:

```
  
  

```

JS:

```
let big = document.querySelector('#bigImg');  
  
function change(file) {  
    big.src = file; // 直接改 src 屬性  
}
```

屬性也能改：除了 `textContent` 和 `style`，`src`，`href`，`id` 都能改！

實作練習：滾動偵測 (SCROLL EVENT)

網頁捲動時，導覽列變色。

```
let nav = document.querySelector('nav');

window.addEventListener('scroll', function() {
  // 取得目前捲軸垂直位置
  let y = window.scrollY;

  if (y > 100) {
    nav.classList.add('active'); // 變色
  } else {
    nav.classList.remove('active'); // 復原
  }
});
```


實作練習：輸入檢查 (VALIDATION)

防止使用者沒填資料就送出。

```
let input = document.querySelector('#username');
let submit = document.querySelector('#submitBtn');

submit.addEventListener('click', function() {
  // 取得輸入框的值 (value)
  let val = input.value;

  if (val === "") {
    alert("請輸入名字!");
    input.style.border = "2px solid red"; // 變紅框警告
  } else {
    alert("歡迎，" + val);
  }
});
```

定時器 (TIMER) - 讓網頁有時間觀念

想做「3秒後自動彈出廣告」？

setTimeout (鬧鐘)

單位是**毫秒** (1000 ms = 1秒)。

```
setTimeout(function() {  
    alert("時間到！");  
}, 3000);
```

用途：廣告彈窗、過場動畫結束後跳轉。

常見錯誤 (DEBUGGING)

如果程式不動：

1. 看 Console 有沒有紅字。

- `Uncaught TypeError: Cannot read property '...' of null`
- 意思通常是：**你抓錯元素了** (ID 打錯字，或者 script 放在 head 裡還沒讀到 body)。

2. 檢查拼字。

- `getElementByid` (錯) -> `getElementById` (對)
- JS 大小寫很敏感！

3. Script 標籤位置。

- 放在 `</body>` 結束標籤的**前一行**最保險。

補充：現代開發的主流 - TYPESCRIPT

雖然我們現在學的是 JavaScript，但一定要知道：
目前業界真正開發時，幾乎都是使用 TypeScript (TS) ！

- **TypeScript 是什麼？**
 - 它是 JavaScript 的**嚴格版** (由 Microsoft 開發)。
 - 加強了型別檢查 (避免你把字串當數字算)。
- **為什麼要用？**
 - JS 太自由容易出錯，TS 能在寫程式碼時就抓出錯誤 (紅字警告)。
- **不用擔心**
 - TS 寫完後，還是會翻譯 (Compile) 成 JS 給瀏覽器看。
 - **只要 JS 基礎打好，學 TS 很快！**

總結

JavaScript 的核心流程就是三步驟：

1. 抓 (Select) : `document.querySelector`
2. 聽 (Listen) : `addEventListener`
3. 改 (Modify) : `textContent` , `style` , `classList`

不用死記語法，只要知道這個邏輯，
你可以做出任何互動網頁！

下週，我們將看看現代網頁開發的神器：前端框架！